XBee Sniffer Interface (XSI) - Documentation

Table of Contents

Preface

This is the official documentation that provides the necessary information to use the XSI application provided for free on Github. Anything not from the original repository is not controlled by us. Content in this document may be subject to change based on content updates to the app, check the header to see what version this document is written under.

Getting Started

The repository on Github should already have a guide on setting up and starting the application. The below instructions are more of a recap if you have not done so already.

Recap: Once you've downloaded the folder and have everything ready and set up (whether it be for either the executable version or Python version), follow the steps below to launch the application:

1. Navigate to the folder you extracted
2. Once inside that folder, extract the bin folder inside to unpack the program
3. Delete the zipped bin folder as it is no longer needed
4. Navigate inside the bin folder
5. Inside the bin folder, you should see one of the following: 'xsi.exe'; 'xsi'; 'xsi.pyw'
6. This is the file that you want to run as this starts the execution of the program
7. Run the file by either double clicking on it, or right clicking and selecting appropriate methods of running it (if using the Python version on unix, it's recommended to run it using the terminal)

NOTE: It is possible to create a shortcut of the executable on any system. Other systems are easier than others though, so if it's not possible, knowing where to execute the program is necessary.

If the app opens up successfully, then you should be greeted with a screen similar to the one shown in Figure 1a and Figure 1b.

Figure 1a: Unannotated view of the app on start

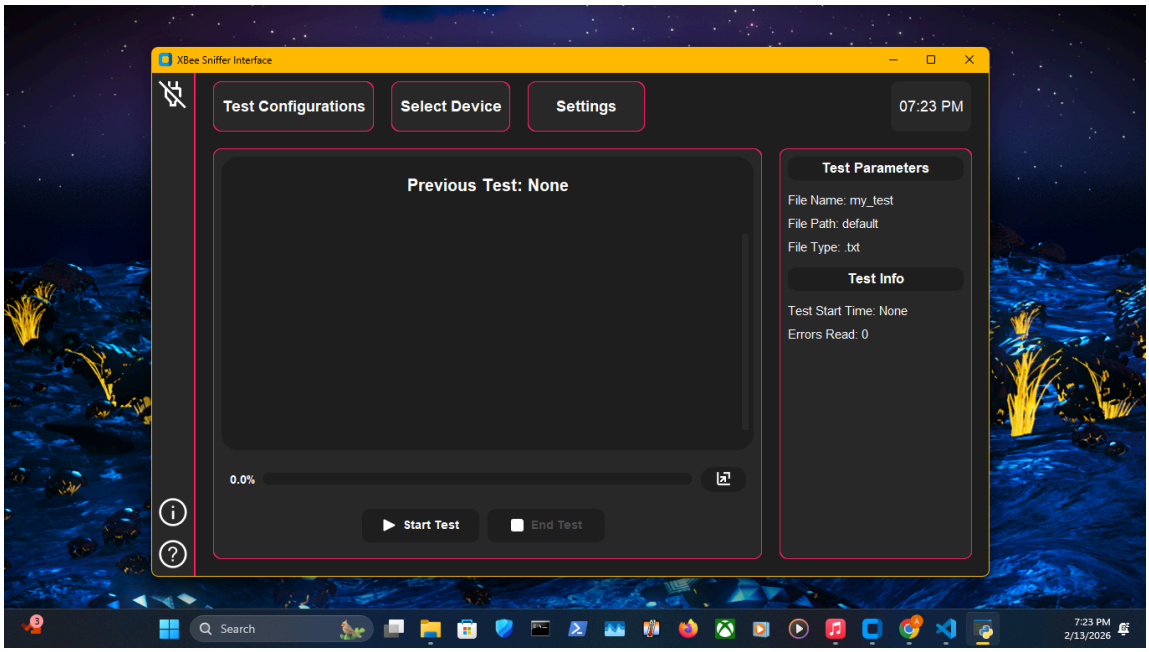
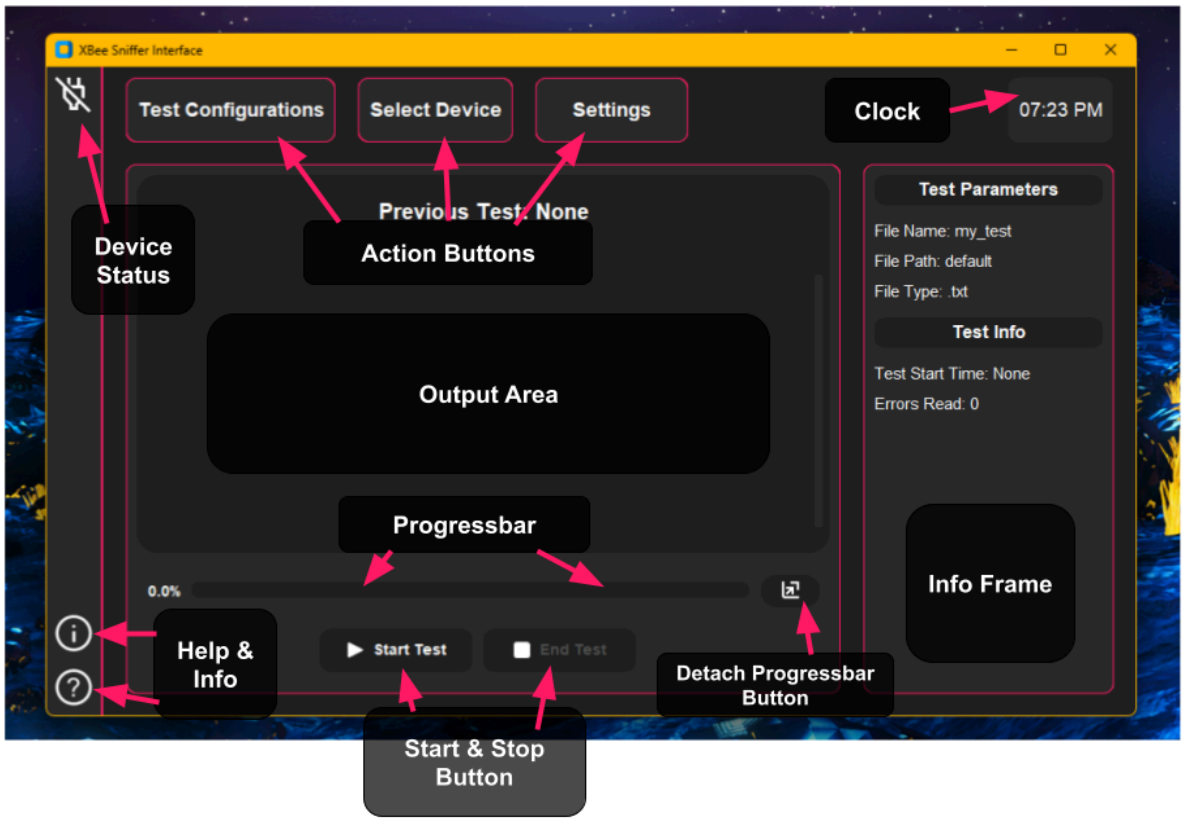


Figure 1b: Annotated view of the app on start

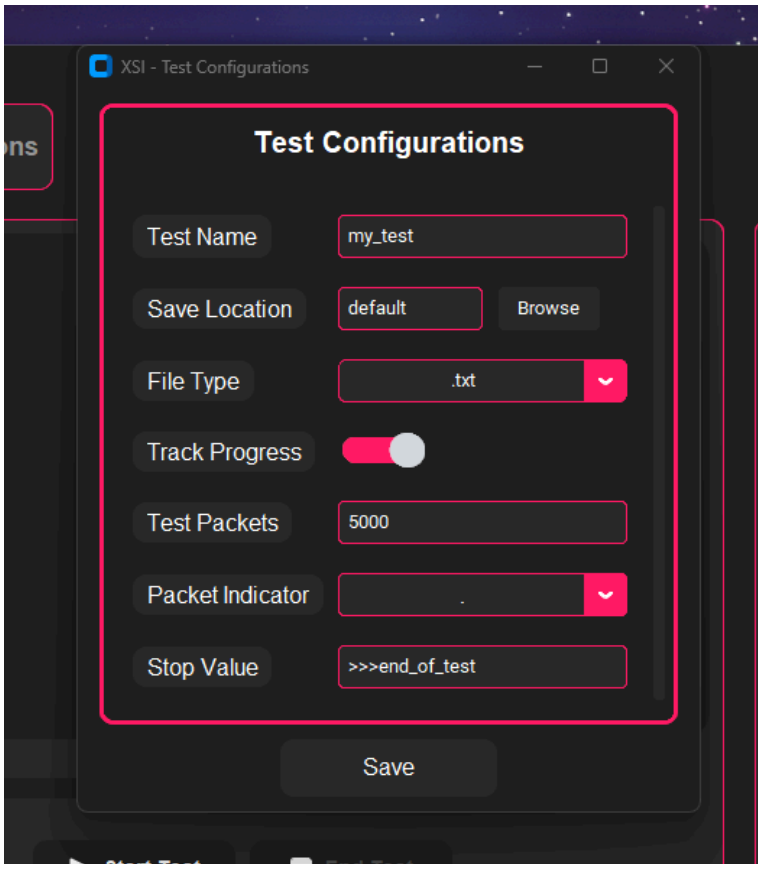


Configurating For Tests

The application makes it super simple to configure for new tests. It includes a variety of options that allow a user to have complete control of what they want their test file to save and tracking test progress.

To change test configurations, simply click on the **'Test Configurations'** button located at the top. Once you've clicked on the button, a new window will appear that looks something like Figure 2 below which shows what the window looks like.

Figure 2: Test configuration window



From Figure 2, we can see a list of options available to change for a test.

- **Test Name**: changes the test and file name of the test
- **Save Location:** path to save location *(this feature cannot be turned off)*
- **File Type:** changes the file extension type, currently only .txt and .csv are supported
- **Track Progress:** Indicates if you want a progress bar during testing
- **Test Packets:** If doing a test that has a set amount of packets, use this to help the progress bar track the progress throughout the test, otherwise progress bar will not work as intended or at all
- **Packet Indicator:** Character to choose how to find what packet the test is currently at

- **Stop Value:** string indicator to end the test automatically, if your base station never prints one, then leave the default value in as it won't end the test anyway

Tracking test progress may be the most confusing part; but with a quick explanation, it's really simple to set up if you have the means to. A base station can send a message out to display what packet it is on; let's use the example string of "**...P3232…**" as it will help demonstrate. If the packet indicator is set to '**.**' (Period), then the app will look for only one of that character in the received data. So if our received data is like the example string, then it will recognize it as an indication of a packet number. Then it will parse the data for the packet number the test is on, and update the progress bar (if enabled).

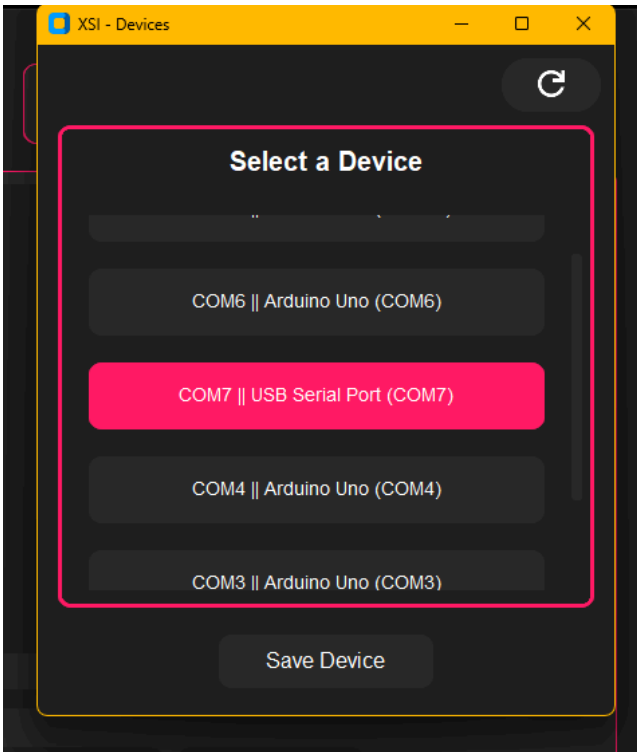
Another thing to look at is the **Save Location** option. By default the app has it set to '**default.**' The default path simply points to the cache folder located inside the bin folder for the app. You can also click the '**Browse**' button to save the file to whatever folder you like.

Once you're done, click the '**Save**' button to save the selected test configurations, and the info frame will update automatically.

Selecting A Device

Choosing a device is a quick process. Start by selecting the '**Select Device'** button at the top. Once clicked, you should be greeted with a window as shown in Figure 3.

Figure 3: Select device window



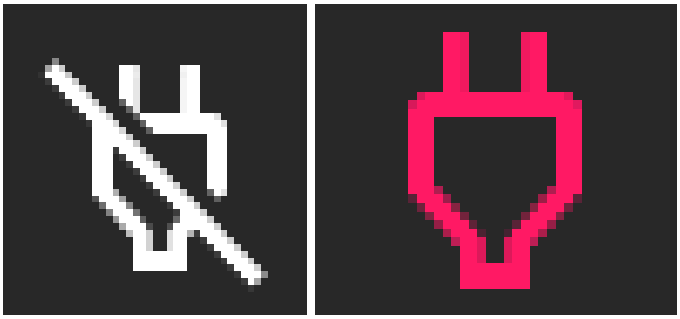
This window allows you to select from a variety of available devices on your computer. However, obviously not all these devices are XBee radios. Usually, your XBee radios will be marked as '**USB Serial Port (COMX).**' Any other devices are likely not going to work. Once you select the device you want, click on the '**Save Device**' button. Another window will appear confirming the device's response. The device response can have one of three states:

- **Loading:** The device is being checked currently
- **OK!:** The device successfully sent back a message and confirmed to be an XBee radio
- **FAILED:** The device has been confirmed to not be an XBee radio

If you receive a response of '**OK,**' then all windows will close automatically. Otherwise, you'll need to troubleshoot which device is actually your XBee radio. You can use the refresh button at the top of the window if you think your device was recognized late.

In the top left corner, the device status will change, depending on what the state of the device is. This is indicated by what icon is currently being displayed. View Figure 4 to see what they mean.

Figure 4: The XBee radio is disconnected (left) and connected (right) to the computer



NOTE: During testing, the status of the XBee radio is checked during data transmission. If the XBee radio is disconnected during any period of time during the test, the test will automatically end and exit with errors. The device status will change back to the one shown in Figure 6 (left).

Starting The Test

Once all configurations and a device has been selected, you are ready to start your test. Once you click on the start test button, the app will essentially go into testing mode. So if you have any configurations or settings you want to change, you can not do so until the test is either ended by the user or ended automatically.

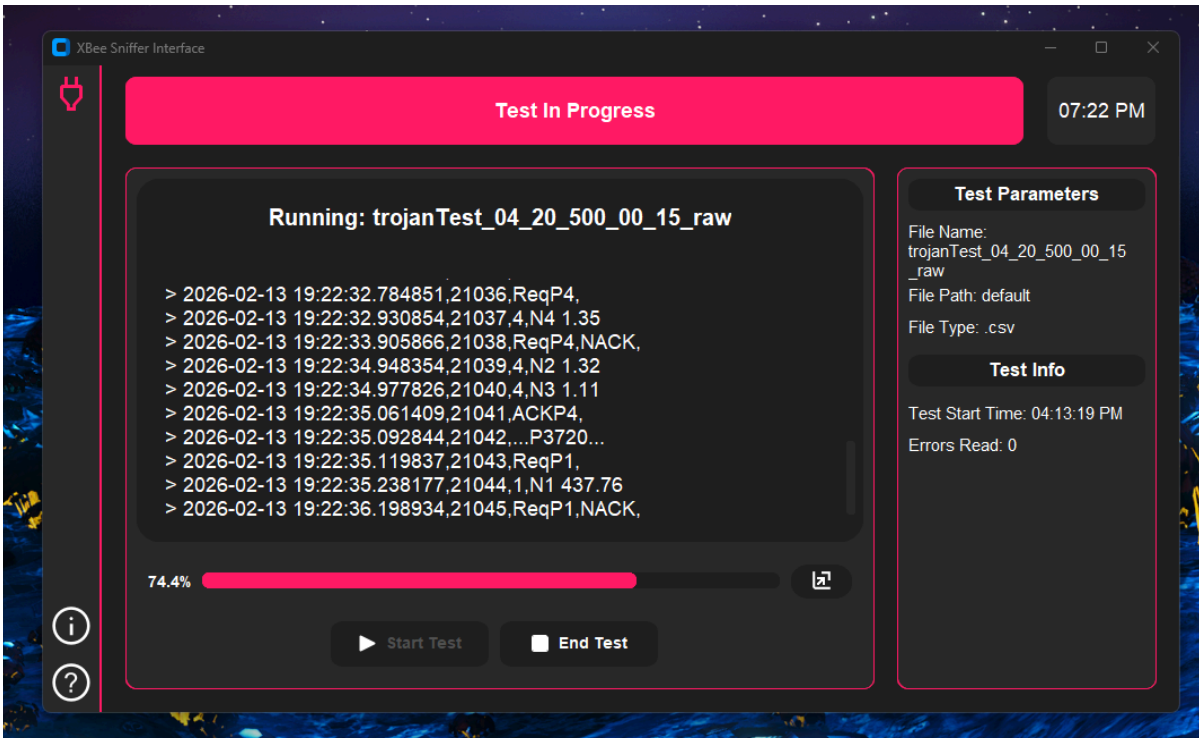
The moment that the start button is clicked, the app immediately starts reading from the selected radio. It is then outputted to app once a piece of data has been read in the following format:

> YYYY-MM-DD HH:MM:SS.XXXXXX,INSTANCE,DATA

Where **INSTANCE** is what item number the test is on, and **DATA** is the data decoded from the radio.

An ongoing test will look something similar to Figure 5 shown below.

Figure 5: An test running

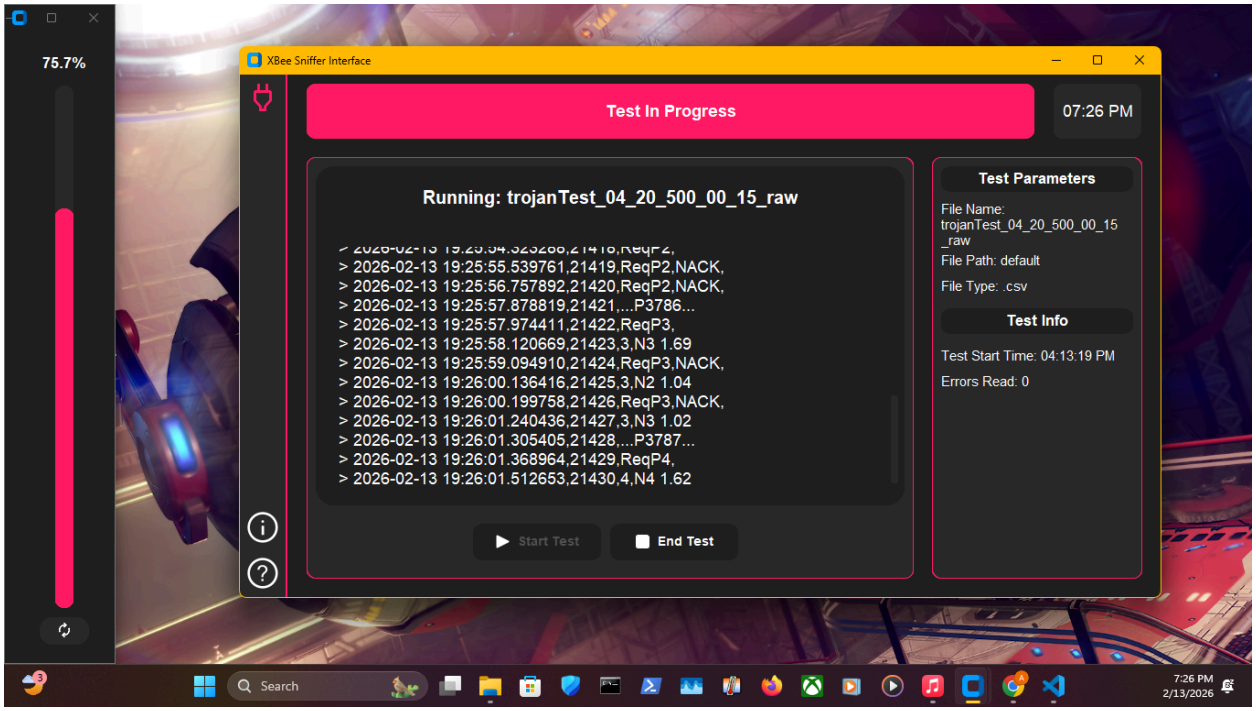


NOTE: The terminal output frame, as of the current version, can only have 35 items in at a time. However, you will be able to view all items in the output file.

WARNING: During testing, DOT NOT unplug your radio device, as this will cause the test to automatically end without warning!

If the progress bar is enabled correctly, you will see it update throughout the test. Alongside this, there is a button to the right of it. This button will actually detach the progress bar and create it as a separate window. See Figure 6 for a clear picture.

Figure 6: Test running with a detached progressbar



In the detached window, there is also a button to flip the progress bar between vertical and horizontal orientations. You might have to extend the window in order to see the button.

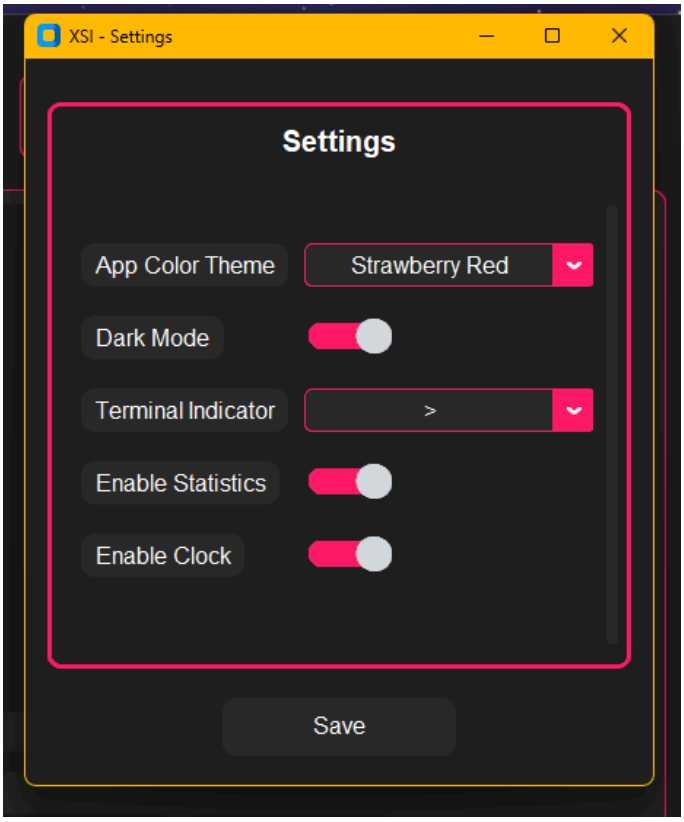
The info frame will update with errors read during reading the stream from radio and say the start time of the test.

If you set up the stop lookout value correctly from the test configuration window, then the test will end automatically, otherwise you will have to use the '**End Test**' button.

Settings

The app has several settings that can be changed while not in testing mode. To open settings, click on the '**Settings**' button at the top. Once clicked, you should see a window similar to the one in Figure 7.

Figure 7: Settings window



From Figure 7, we can see that the following options are available:

- **App Color Theme:** Changes the secondary color of the app theme
- **Dark Mode:** Changes the background to a white / grey color instead of dark grey, the theme color stays the same
- **Terminal Indicator:** Changes the newline indicator of the terminal
- **Enable Statistics:** Enables or disables the info frame on the right side
- **Enable Clock:** Enables or disables the clock that can be found in the top right corner of the app *(This does not mean that the app will stop reading all time together)*

Most of the settings are self explanatory, mess around with them before activating a full test, as you will not be able to change them during testing mode.

Info And Help

If you take a look back at Figure 1b, you will see in the device frame, a help and information button. These don't do that much. The information button will open up a new window that gives detail about the application and quick information like version, app name, what it's built with, etc. This can be opened during testing mode as well, as this window does not have any affect on testing. The help button will simply open up this pdf in the set default application on your device.

If you need more information or help, try reading the Github page or this document as they provide the most information on the application.

Thank You

Thank you for downloading and using the application, as this was originally just a passion project. Hopefully, this application makes your XBee radio experience much more pleasant.